Enterprise Customer Onboarding Agent — Solution Design Document

April 2026

1. Architecture Overview

The solution implements an **AI-powered automation agent** that orchestrates enterprise customer onboarding from deal closure through SaaS provisioning. Built with **Pydantic AI** for agentic tool calling and **FastAPI** for the REST interface, the agent uses native LLM tool calling to autonomously reason through 21 registered tools. Each tool is mirrored as a **FastMCP** server definition for MCP-style extraction. A second **CS Assistant Agent** shares all tools but returns free-form text for conversational interaction via a **Streamlit UI**. Data flows sequentially across systems via **cross-system chaining**: Opportunity.ContractId → SF Contract.Id → CLM.salesforce_contract_id → Invoice.clmContractRef, with fallback to account-based lookups when chain links are missing.

1.1 System Integrations

System	Objects & Key Fields
Salesforce CRM	`Account` (Id, Name, IsDeleted, OwnerId), `Opportunity` (StageName, Amount, **ContractId**), `Contract` (**Id**, Status, StartDate), `User` (IsActive, Email). [API Ref]
CLM System	contract_id, status, **salesforce_contract_id** (chains from SF Contract.Id), signatories, key_terms. Mock simulating DocuSign CLM.
NetSuite ERP	`Invoice` (status, dueDate, amountRemaining, total, **clmContractRef** chains from CLM contract_id). [API Ref]
Provisioning	Tenant creation + 14-task onboarding checklist with dependencies, owners, and due dates.
Currency (Frankfurter)	Live and historical exchange rates via ECB data. Converts invoice amounts across currencies (e.g., CAD↔USD) using the invoice date rate. [API Ref]

Field Selection Rationale: `Account.IsDeleted` validates account exists; `Opportunity.StageName="Closed Won"` confirms deal closure; `Invoice.status` identifies payment blockers; `CLM.status` in {EXECUTED, SIGNED} required for provisioning. Currency conversion uses historical rates from the invoice date for financial accuracy. Minimizes API calls while capturing decision-critical data.

1.2 Data Flow

(1) Webhook with `account_id` + `correlation_id` **(2)** Fetch account, then user/opportunity in parallel; chain sequentially: Opp.ContractId → SF Contract → CLM (via salesforce_contract_id) → Invoice (via clmContractRef) **(3)** `validate_business_rules` + `check_financial_alignment` **(4)** PROCEED → provision + monitor; BLOCK/ESCALATE → notify.

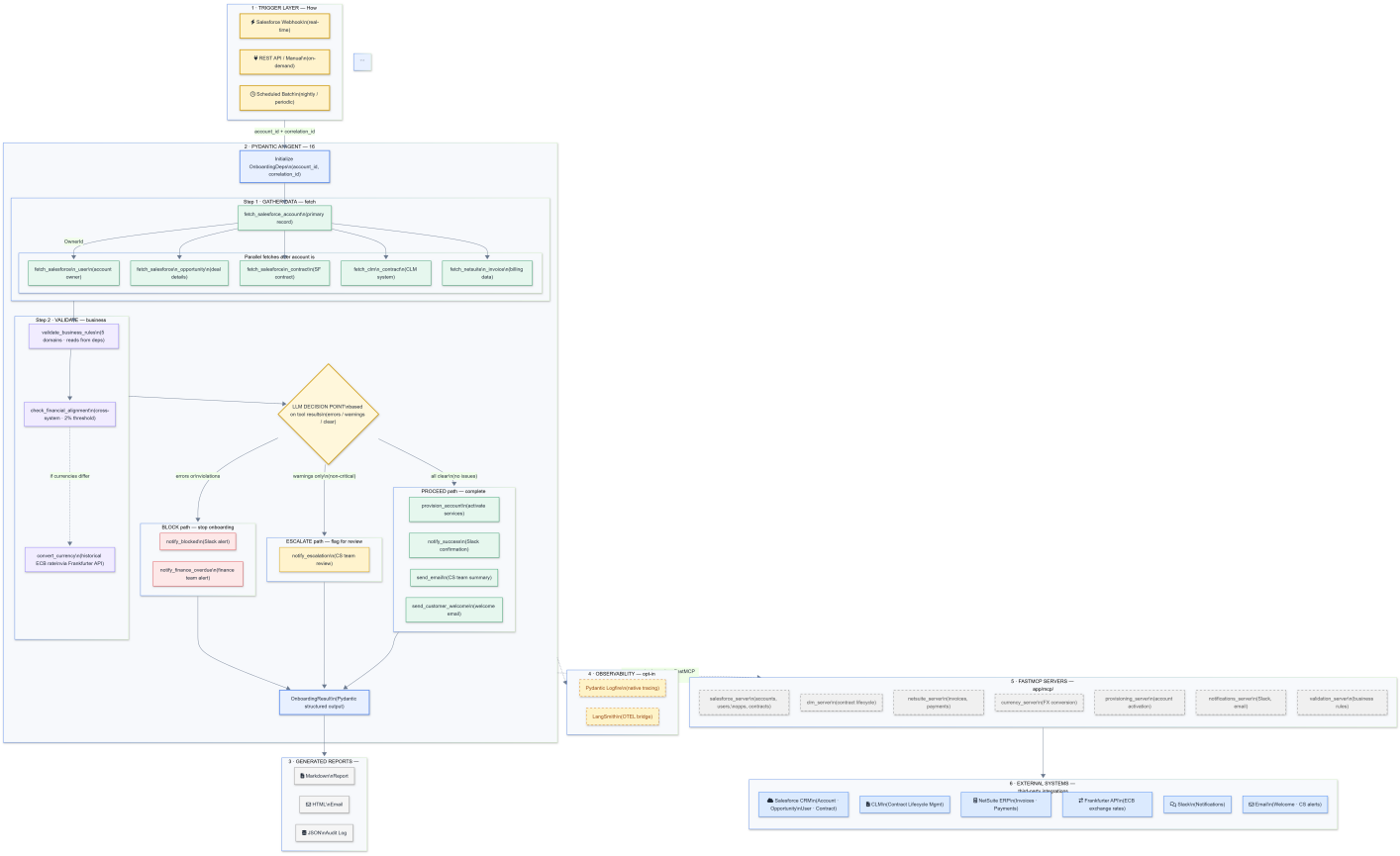

Figure 1: Architecture: triggers, Pydantic AI agent orchestration with 21 tools, cross-system data chaining, FastMCP servers, and external system integrations.

2. AI Agent Application

The agent leverages **OpenAI GPT-4o-mini** (configurable via `OPENAI_MODEL`) with **Pydantic AI** for native tool calling. The LLM autonomously decides which of the 21 registered tools to call, in what order, and how many times. Falls back to **Ollama** local models when no API key is available. A second **CS Assistant Agent** shares all tools with free-form text output for conversational monitoring via a **Streamlit UI** (Dashboard, Run Onboarding, Chat).

2.1 LLM-Powered Intelligence

- **Risk Analysis**: Evaluates API errors, violations, warnings → risk levels (low/medium/high/critical) with business impact and resolution time estimates.
- **Summary Generation**: Human-readable reports for CS teams, translating technical states to actionable insights.
- **Action Recommendations**: Prioritized steps with owner assignment (CS, Finance, IT/DevOps, Legal, Sales Ops).
- **Error Interpretation**: Converts technical codes (e.g., `INVALID_SESSION_ID`) to plain English with resolution steps and responsible teams.

2.2 Autonomous Actions

Upon decision, the agent performs the following autonomous actions:

(1) Auto-provisions the SaaS tenant with tier-appropriate configuration (Enterprise/Growth/Starter).

(2) Creates a 14-task onboarding checklist with dependencies, ownership (system/cs_team/customer), and due dates.

(3) Sends Slack notifications to `#cs-onboarding-alerts` (blocked) or `#cs-onboarding` (success/escalate).

(4) Sends customer-facing welcome emails with tenant ID and login URL.

(5) Generates HTML/Markdown/JSON reports for auditing and traceability.

(6) Post-provisioning: monitors progress, detects risks, sends reminders, escalates stalled onboardings.

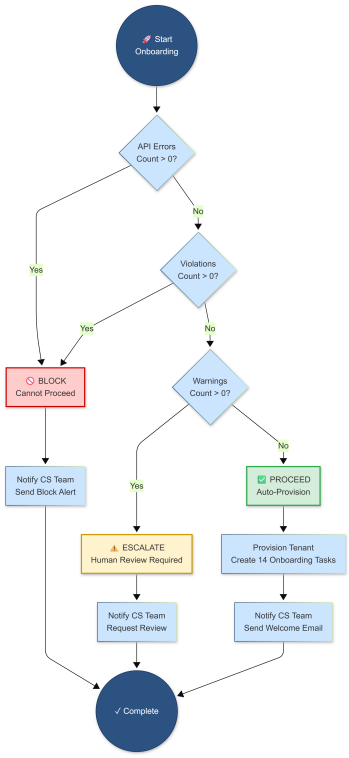

Figure 2: Decision: API Errors/Violations → BLOCK, Warnings → ESCALATE, Clear → PROCEED.

2.3 Onboarding Task Management

The provisioning system creates a 14-task onboarding checklist with the following structure:

Task Categories:

- **Automated (4)**: Tenant creation, API credentials, welcome email, training materials
- **CS Team (5)**: Schedule/conduct kickoff, SSO config, custom reports, 30-day check-in
- **Customer (4)**: Verify login, complete tour, invite team, create first campaign
- **Milestone (1)**: Onboarding complete marker

API Endpoints:

- `GET /tasks/{id}` – All tasks with status
- `GET /tasks/{id}/pending?owner=cs_team` – Filtered pending
- `GET /tasks/{id}/overdue` – Overdue task alerts
- `PUT /tasks/{id}/{task_id}` – Update task status
- `GET /tasks/{id}/next-actions` – Ready-to-execute tasks

Tasks include dependencies (e.g., "Complete Tour" depends on "Verify Login"), due dates calculated relative to provisioning, and ownership assignment enabling filtered views per team.

2.4 Business Rule Validation (Invariants)

The agent enforces a two-tier validation system across five domains. **Violations** are blocking (cause BLOCK decision); **Warnings** are non-blocking (cause ESCALATE for human review).

Domain	Blocking Violations	Non-blocking Warnings
Account	Missing Id/Name, IsDeleted=true	Missing BillingCountry, Industry, OwnerId
Opportunity	Stage ≠ "Closed Won", Missing Id/AccountId	Missing Amount, CloseDate, ContractId
Contract (CLM)	Status ∉ {EXECUTED, SIGNED}, Missing contract_id	No effective/expiry date, pending signatories
Invoice	Voided, Cancelled, Missing invoice_id	Overdue, Pending, <50% paid, missing due_date
User	Inactive, Missing Id/Email/ProfileId	Missing Title, Department, ManagerId

3. Orchestration & Event-Driven Flows

Trigger	Source	Endpoint
Webhook	Salesforce Flow	`POST /webhook/onboarding`
Manual	CS Team	`POST /demo/run/{account_id}`
Batch	Scheduler	`POST /demo/run-all`

Error Simulation: `/demo/enable-random-errors` injects failures (401 auth, 400 validation, 429 rate limit, 500 server) at configurable rates for chaos testing without modifying business logic.

Report Generation: Each run produces three output files:

- `run_report_*.md` – Full Markdown report with all details
- `email_*.html` – Professional HTML email (blocked, escalation, success, or welcome)
- `audit_*.json` – Machine-readable audit log

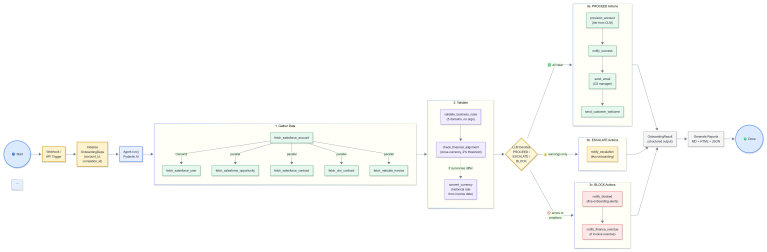

Figure 3: Agent workflow: full lifecycle from webhook ingestion through tool calling, validation, and decision routing to completion.

3.1 API Error Classification & Handling

Each integration returns structured error payloads with `error_type`, `error_code`, `http_status`, `message`, `resolution`, and `owner`:

Error Type	HTTP	Resolution
Authentication	401	Re-authenticate, refresh token (IT/DevOps)
Authorization	403	Check profile/permission sets (System Admin)
Validation	400	Fix field values in source (Data Admin)
Rate Limit	429	Implement backoff, increase quota (Integration Admin)
Server Error	500	Check status page, retry later (Support Team)

Error Handling Features:

- **In-place Error Simulator**: `ERROR_SIMULATOR` object modified in-place when enabled; all modules reference the same instance
- **Comprehensive Catching**: Each integration catches specific error types AND generic `APIError` as fallback
- **Error-Aware Decisions**: `make_decision` checks `api_errors` first — system failures block onboarding
- **Full Traceability**: API errors added to violations and appear in reports with context

4. Trade-offs, Assumptions & Considerations

4.1 Key Trade-offs

Decision	Trade-off
API Errors → BLOCK	Integrity over speed
Sequential Data Fetching	Clearer failure tracing but has higher latency
Rule-based Fallback	Less smart but reliable
14 Fixed Tasks	Predictable but rigid
LLM Advisory Only	Auditable but less adaptive

4.2 Assumptions

- Salesforce is source of truth for accounts
- CLM status reflects actual signatures
- Invoice payment status is current
- Invoices may be in different currencies (e.g., CAD, USD); cross-currency comparison uses historical ECB rates from the invoice date with a 2% alignment threshold
- One invoice per account per onboarding run
- One Opportunity per Account for onboarding (no multi-deal scenarios)
- Contract has a single version with no amendments
- CS monitors Slack channels for alerts
- Single onboarding per account at a time
- Customer email available in CLM signatories
- Account ID is consistent across all systems

4.3 Limitations

- **Mock integrations**: Salesforce, NetSuite, and CLM use simulated data; not connected to real APIs
- **In-memory state**: Provisioning and task data lost on restart; no persistent database
- **Single instance**: No horizontal scaling or load balancing for high-volume webhooks
- **No approval UI**: ESCALATE decisions notify Slack but lack a UI for human approve/reject workflow
- **No webhook auth**: Inbound webhooks not verified via HMAC signature or shared secret

4.4 Testing & Demonstration

The system includes 7 pre-configured demo scenarios validating all decision paths, including cross-currency financial alignment:

Account ID	Scenario	Description	Decision
`ACME-001`	Happy Path	All checks pass, contract executed, invoice paid	PROCEED
`BETA-002`	Opportunity Blocked	Opportunity in "Negotiation" stage	BLOCK
`GAMMA-003`	Overdue Invoice	Invoice 25+ days overdue, needs finance review	ESCALATE
`DELETED-004`	Deleted Account	Account marked `IsDeleted=true`	BLOCK
`MISSING-999`	Account Not Found	Account does not exist in any system	BLOCK
`FOREX-005`	FX Invoice Mismatch	CAD $145k invoice vs USD $100k opportunity; historical rate conversion reveals gap >2%	ESCALATE
`PARTIAL-006`	Partial Payment Gap	$190k paid of $200k invoice (5% underpayment exceeds 2% threshold)	ESCALATE

4.5 Observability & Tracing

The agent supports **dual opt-in tracing** — both can run simultaneously:

- **Pydantic Logfire**: Native Pydantic AI agent tracing — full reasoning chains, tool calls with inputs/outputs, structured output validation. Dashboard at `logfire.pydantic.dev`.
- **LangSmith**: Agent execution runs, tool invocations, LLM completions via OpenTelemetry bridge (`OtelSpanProcessor`). Logfire owns the TracerProvider; LangSmith attaches as a span processor.
- **Structured JSON Logging**: All events logged with `correlation_id`, timestamps, and context. Logs written to `logs/` directory for audit trails.

4.6 Security Patterns

All integrations are mocked but demonstrate production security patterns:

- **OAuth Simulation**: Token expiry and refresh flows with distinct error types for expired vs invalid tokens
- **Permission Checking**: Role-based access control verified before every API call
- **Credential Validation**: Structured error hierarchy (`AuthenticationError`, `AuthorizationError`, `ValidationError`, `RateLimitError`)
- **Audit Logging**: Correlation IDs propagated end-to-end for traceability across all systems
- **Error Masking**: No sensitive data (tokens, secrets) exposed in API responses or generated reports

5. Multi-Agent Architecture via Model Context Protocol (MCP) Server Connections

Note: All 21 agent tools are mirrored as FastMCP server definitions in `app/mcp/` (7 servers: Salesforce, CLM, NetSuite, Currency, Provisioning, Notifications, Validation). These can be extracted to standalone MCP services for multi-agent collaboration.

The architecture supports multi-agent collaboration through the **Model Context Protocol (MCP)**, enabling specialized agents to interact through standardized, governed tool interfaces rather than direct API access. Each integration is wrapped as a FastMCP server, cleanly separating decision logic from execution concerns while allowing agents to remain focused on their respective domains.

Agent	Responsibility & MCP Tools
Coordinator	Orchestrates end-to-end workflow (`salesforce.*`, `provision.*`)
Contract Agent	Monitors contract execution and signatures (`clm.get_contract`)
Finance Agent	Evaluates billing, payments, and cross-currency alignment (`netsuite.*`, `currency.*`)
Task Monitor	Detects overdue onboarding tasks (`tasks.get_overdue`)

Benefits: Independent versioning per integration • Clear domain ownership • Centralized access control and audit logging • Scalable agent complexity • Cross-agent context sharing.

Concurrency & API Execution Strategy: MCP servers would act as controlled execution layers for external APIs, enabling concurrent and batch-aware request handling. Where supported, batch or composite API calls are leveraged to reduce network overhead and latency (e.g., Salesforce composite reads). If batch execution fails or yields partial results, the system safely falls back to bounded parallel single-entity requests.

Concurrency limits, rate limiting, and circuit breakers would be enforced at the MCP layer to prevent downstream system overload and cascading failures. This approach improves onboarding throughput while preserving idempotency, error isolation, and predictable failure handling across agents.

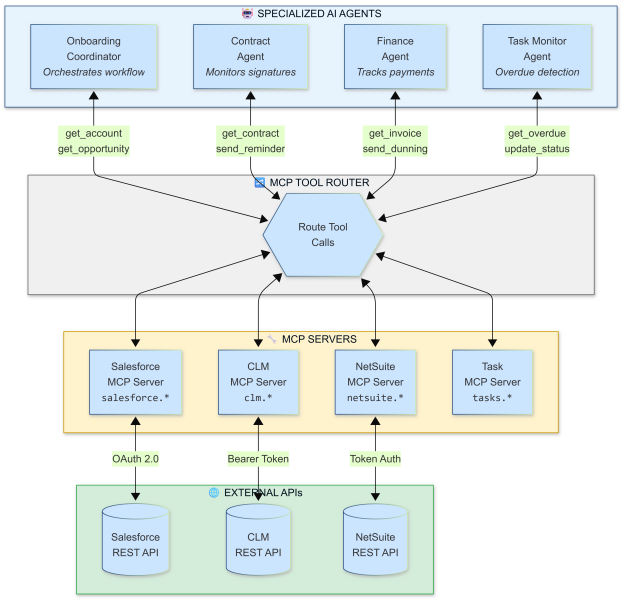

Figure 4: MCP: Specialized agents collaborate through shared execution servers wrapping external APIs.

6. Production Considerations

For additional production enhancements, see the `production_roadmap` directory in the repository.

Cloud Deployment

Frontend: A **Streamlit UI** is currently implemented with three pages: Dashboard (all results by decision with health/progress), Run Onboarding (scenario runner), and Chat with Agent (conversational CS assistant). For production, migrate to a role-based interface (React, ASP.NET) with authentication, WebSocket updates, and RBAC.

AWS Infrastructure: The onboarding agent executes as containerized services on ECS/Fargate, enabling horizontal scaling based on inbound webhook volume. LLM inference is handled via **AWS Bedrock** rather than direct OpenAI calls, allowing multi-model flexibility (Claude, Llama, Titan), private networking with traffic contained within the VPC, IAM-based authentication, and enterprise-grade security and compliance. Bedrock enables cost controls and model switching without application changes.

Docker + Kubernetes: Containerization ensures consistent environments across development, staging, and production. Kubernetes orchestration (or ECS equivalents) provides auto-scaling for traffic spikes (e.g., batch onboarding runs), self-healing services, rolling updates, and zero-downtime deployments as agent logic evolves.

CI/CD & Observability

CI/CD (GitHub Actions / CodePipeline): Automated pipelines validate agent logic on every pull request, including unit tests for invariant checks, risk analysis, and decision routing. Deployments are reproducible and auditable, with fast feedback loops, reduced manual error, and the ability to roll back safely in case of regressions impacting onboarding outcomes.

Dual Tracing (Implemented): The agent supports opt-in dual observability: **Pydantic Logfire** for native Pydantic AI agent tracing (reasoning chains, tool calls, structured output validation) and **LangSmith** via an OpenTelemetry bridge (`OtelSpanProcessor`) for LLM execution runs, token usage, and latency. Both can run simultaneously — Logfire owns the TracerProvider and LangSmith attaches as a span processor.

Monitoring & Alerting: Prometheus metrics capture agent throughput, API error rates, and decision distributions. Distributed tracing via DataDog or Jaeger enables end-to-end visibility across webhook ingestion, integrations, LLM calls, and provisioning. PagerDuty alerts notify on-call teams of critical failures such as sustained API outages or elevated BLOCK rates.

Agentic Execution & API Governance

Deterministic Decision Boundaries: The onboarding agent uses an agentic architecture where the LLM reasons through tools via native tool calling (Pydantic AI `@agent.tool` decorators). However, deterministic business rules enforced by `validate_business_rules` and `check_financial_alignment` tools define all violations and warnings. The LLM decides the routing (`PROCEED`, `ESCALATE`, `BLOCK`) based on these tool outputs, ensuring predictable, testable, and auditable outcomes.

Cross-System Data Chaining: The agent fetches the Salesforce account first, then user/opportunity in parallel. Subsequent fetches chain sequentially: Opportunity.ContractId → SF Contract → CLM (via salesforce_contract_id) → Invoice (via clmContractRef). Fallback to account-based lookups if chain links are missing. For production, a tiered optimization strategy applies: **(1)** Batch API requests where supported (e.g., Salesforce Composite API); **(2)** Concurrent fetches for systems that don't support batching; **(3)** Bounded retry logic with exponential backoff (max 3 retries, 1s/2s/4s delays) for transient failures.

Idempotency & Correlation-Based Execution: Each onboarding run is keyed by a `correlation_id` propagated across webhook ingestion, integrations, logs, reports, and notifications. This enables safe replays, deduplication of duplicate webhook events, and re-entrant execution without risk of duplicate provisioning, notifications, or side effects.

API Error Classification & Retry Strategy: External API failures are classified into retryable (e.g., transient 5xx, rate limiting) and non-retryable (e.g., validation, authorization) categories. Retryable errors follow bounded exponential backoff, while non-retryable errors immediately halt onboarding and surface actionable diagnostics. Circuit breakers prevent cascading failures across dependent systems.

Rate Limiting & Backpressure: Inbound webhook traffic is protected via per-source rate limits and queue-based ingestion to absorb traffic spikes from batch updates or retries. Concurrency limits are enforced per integration to prevent overwhelming Salesforce, CLM, or NetSuite APIs during peak load.

Agent Versioning & Safe Rollouts: Every onboarding run records the agent version, rule set, and prompt version used. This enables full auditability, regression analysis, and controlled rollouts (e.g., canary deployments) of agent logic and prompt updates without impacting in-flight executions.

LLM Data Governance & Security: All data sent to LLMs is field-filtered and sanitized. PII is masked prior to inference, and credentials are never exposed beyond integration boundaries. Trust boundaries are strictly enforced at the API layer.

Multi-Agent Interoperability: All 21 agent tools are mirrored as FastMCP server definitions (7 servers in `app/mcp/`), separating agent logic from external system access. These can be extracted to standalone MCP services with independent versioning, access control, and audit logging. The A2A protocol provides a standardized layer for agent-to-agent collaboration without exposing internal state across boundaries.